

Weather Prediction

Machine Learning & Deep Learning Based Weather Type Classifier

1. Introduction

Weather forecasting plays a crucial role in agriculture, transportation, disaster management and daily planning. Predicting the correct weather condition from measurable atmospheric factors such as temperature, humidity, wind speed and atmospheric pressure can help automate early warnings and decision-making.

In this project, a classification system is developed to predict the Weather Type (Cloudy, Rainy, Snowy or Sunny) using historical weather data. The project follows a complete workflow — data cleaning, outlier removal, exploratory data analysis, a Scikit-Learn preprocessing pipeline, comparison of five classical Machine Learning models and finally a custom-built Artificial Neural Network (ANN) using TensorFlow/Keras that is evaluated in depth and used for the final prediction.

2. Problem Statement

The objective of this project is to build a classification model that predicts the weather type using the following features:

- Temperature (°C)
- Humidity (%)
- Wind Speed (km/h)
- Precipitation (%)
- Cloud Cover
- Atmospheric Pressure (hpa)
- UV Index
- Visibility (km)

Using these features, the model predicts one of four Weather Type classes:

- Cloudy
- Rainy
- Snowy
- Sunny

3. Dataset Overview

The dataset contains 13,200 rows and 11 columns describing recorded weather conditions and the corresponding Weather Type label.

Feature Name	Description
Temperature (°C)	Recorded air temperature
Humidity (%)	Relative humidity in the air
Wind Speed (km/h)	Speed of wind
Precipitation (%)	Chance / amount of precipitation
Cloud Cover	Categorical sky condition (clear, overcast, partly cloudy, etc.)
Atmospheric Pressure (hpa)	Barometric pressure
UV Index	Ultraviolet radiation index
Season	Season of the record (dropped before modeling)
Visibility (km)	Visibility distance
Location	Terrain type of the location (dropped before modeling)
Weather Type	Target variable (Cloudy / Rainy / Snowy / Sunny)

4. Import Libraries, Load Dataset & Data Cleaning

The dataset is loaded using pandas and columns are renamed with proper units for clarity. Initial inspection is performed to identify missing values, duplicate rows and dataset structure.

a. Import Libraries & Load Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('ggplot')

JP = pd.read_csv("weather_classification_data.csv")
JP = JP.rename(columns={'Temperature': 'Temperature (°C)', 'Humidity': 'Humidity (%)',
                        'Wind Speed': 'Wind Speed (km/h)',
                        'Atmospheric Pressure': 'Atmospheric Pressure (hpa)'})
```

b. Identify Missing Values & Duplicates

Column	Missing Values
All 11 columns	0

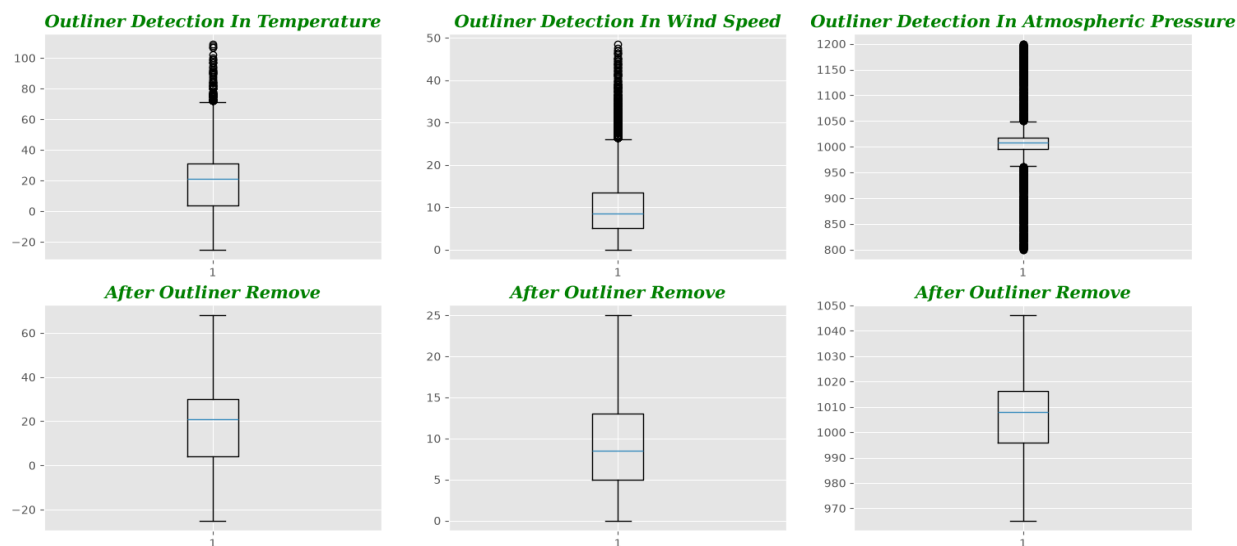
Duplicate rows found: 0 | Original dataset shape: (13,200, 11)

c. Outlier Detection & Removal (IQR Method)

Numeric columns often contain extreme values caused by noise. The Interquartile Range (IQR) method is applied to Temperature, Wind Speed and Atmospheric Pressure to detect and remove such outliers and the results are visually verified using box plots before and after cleaning.

```
Q1 = JP['Temperature (°C)'].quantile(0.25)
Q3 = JP['Temperature (°C)'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.4 * IQR
JP = JP[(JP['Temperature (°C)'] > lower) & (JP['Temperature (°C)'] < upper)]

# The same IQR logic is repeated for Wind Speed (km/h) and
# Atmospheric Pressure (hpa), each with its own bounds.
```



After removing outliers across all three numeric columns, the cleaned dataset is carried forward into feature selection and preprocessing.

5. Feature Selection and Preprocessing Pipeline

a. Dropping Non-Predictive Columns

The Location and Season columns were dropped, keeping the model focused on measurable atmospheric conditions rather than categorical context that does not generalize well.

```
JP = JP.drop(columns=['Location', 'Season'])
```

b. Splitting Features (X), Target (Y) & Label Encoding

```
x = JP.drop(columns='Weather Type')
y = JP['Weather Type']

from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
y = le.fit_transform(y)    # Cloudy=0, Rainy=1, Snowy=2, Sunny=3
```

c. Train / Validation / Test Split

The data is split into training (70%), validation (15%) and test (15%) sets using stratified sampling so that class proportions are preserved across all three sets.

```
from sklearn.model_selection import train_test_split

x_train, x_data, y_train, y_data = train_test_split(
    x, y, test_size=0.3, stratify=y, random_state=56)

x_val, x_test, y_val, y_test = train_test_split(
    x_data, y_data, test_size=0.5, stratify=y_data, random_state=56)
```

d. Building a Column-wise Preprocessing Pipeline

A Scikit-Learn ColumnTransformer combined with Pipeline objects handles numeric and categorical columns consistently:

- Numeric columns — scaled using StandardScaler.
- Categorical columns (Cloud Cover) — encoded using OneHotEncoder

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_col = x.select_dtypes(include=['int64', 'float64']).columns
categorical_col = x.select_dtypes(include=['str']).columns

numeric = Pipeline([('scaler', StandardScaler())])
categorical = Pipeline([('encoder', OneHotEncoder(handle_unknown='ignore'))])

preprocessing = ColumnTransformer([('Numeric', numeric, numeric_col),
                                   ('Categorical', categorical, categorical_col)])

preprocessing.fit(x_train)
x_train_scaled = preprocessing.transform(x_train)
x_val_scaled = preprocessing.transform(x_val)
x_test_scaled = preprocessing.transform(x_test)
```

Resulting shape of the encoded training set: (8205, 11) — 7 scaled numeric columns + 4 one-hot encoded Cloud Cover columns.

This pipeline-based approach ensures the same preprocessing logic is applied consistently to training, validation and test data, and prevents data leakage.

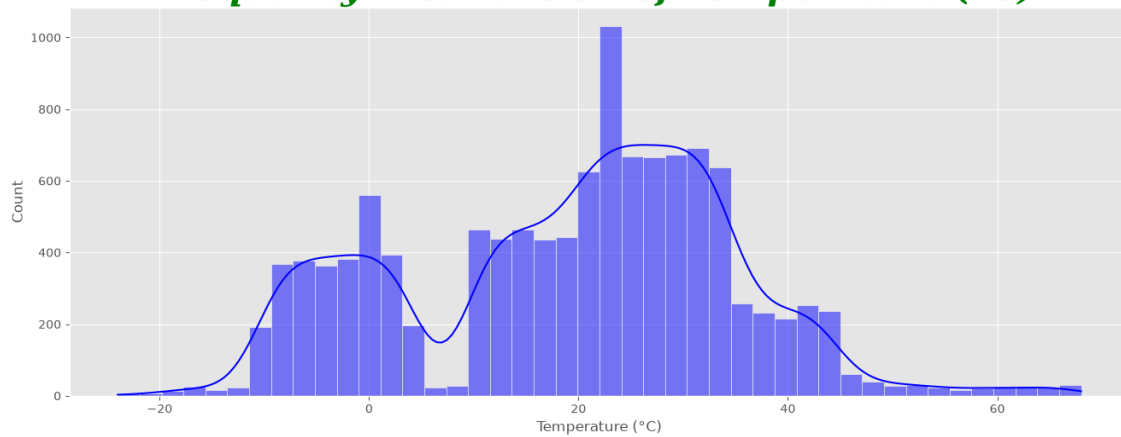
6. Exploratory Data Analysis (EDA)

Several visualizations are generated to understand feature distributions and their relationship with Weather Type.

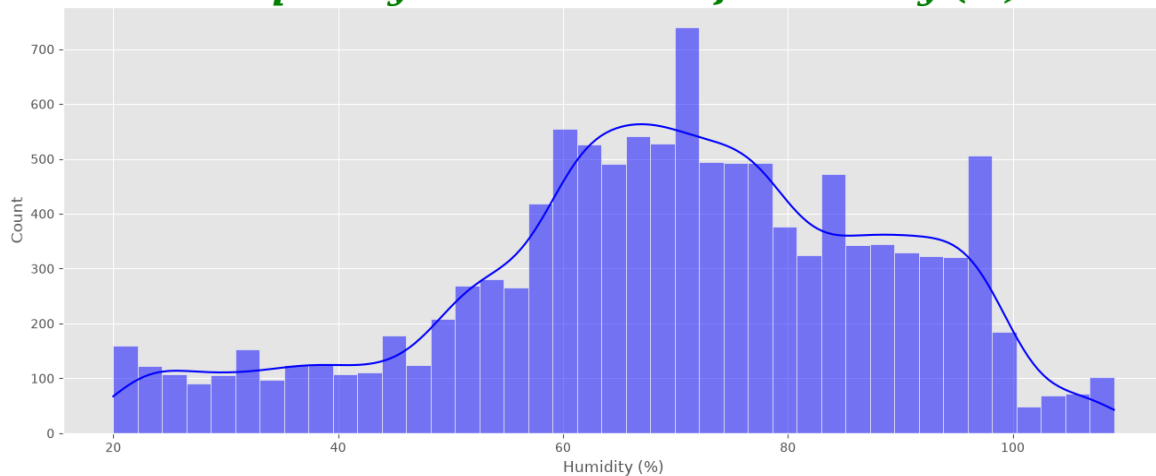
- Histograms with KDE — plotted for every numeric column to view its distribution shape.
- Correlation heatmap — checks linear relationships between numeric features.
- Scatter plots — Temperature vs Humidity, Wind Speed vs Precipitation, and UV Index vs Visibility, each colored by Weather Type.
- Pie chart — visualizes the overall distribution of Weather Type classes.

a. Frequency Distributions (Histograms)

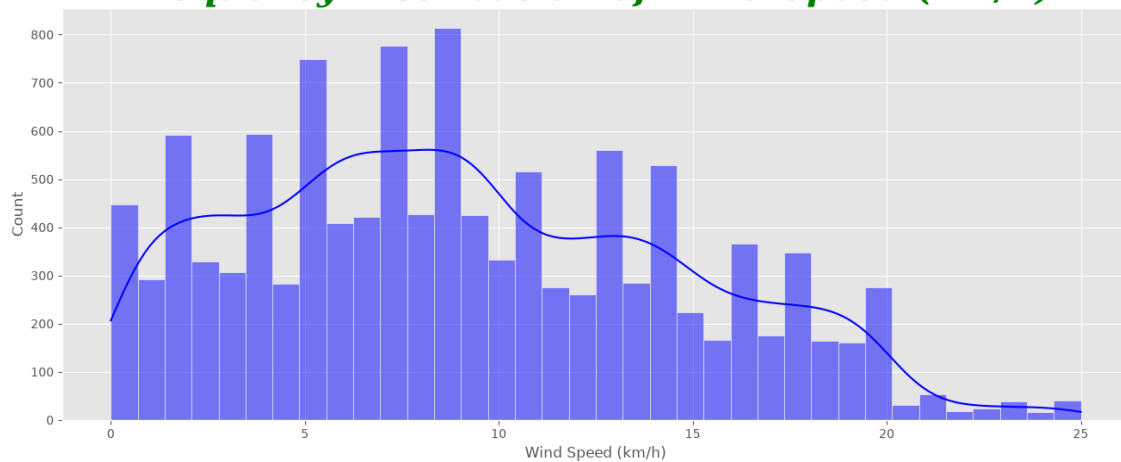
Frequency Distribution of Temperature (°C)



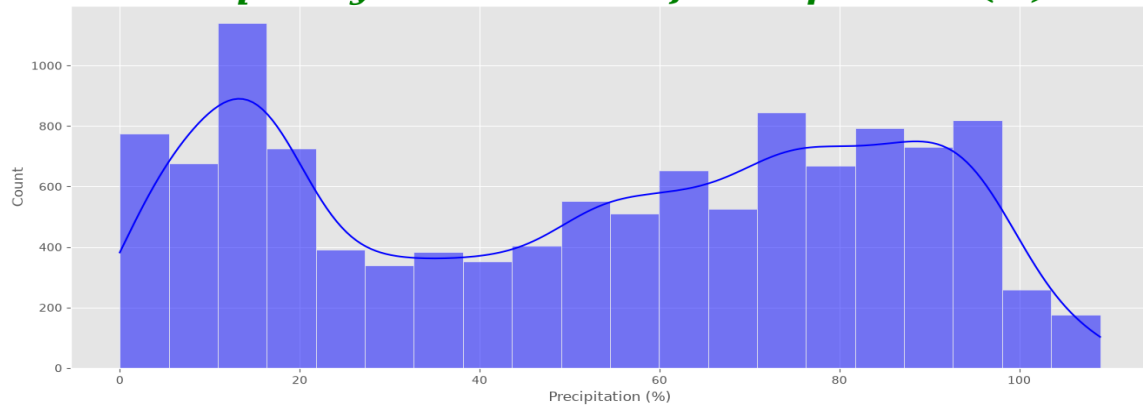
Frequency Distribution of Humidity (%)



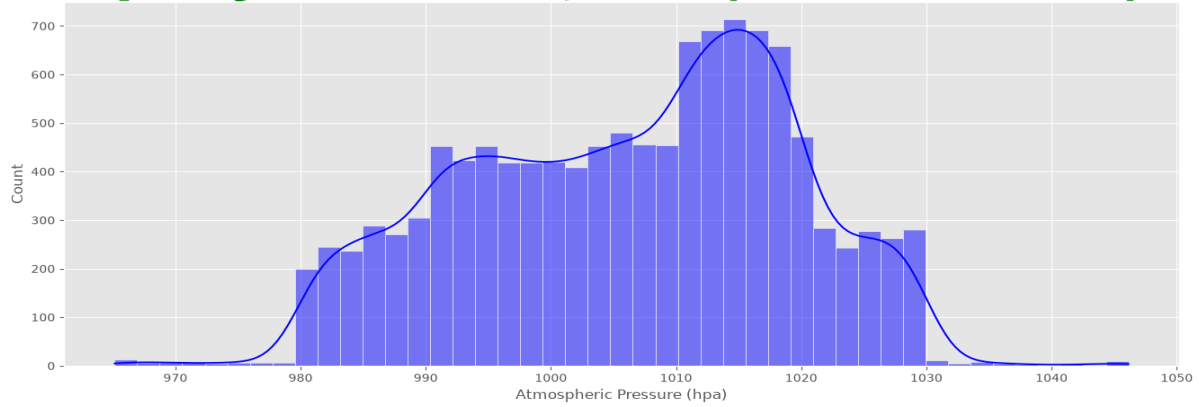
Frequency Distribution of Wind Speed (km/h)



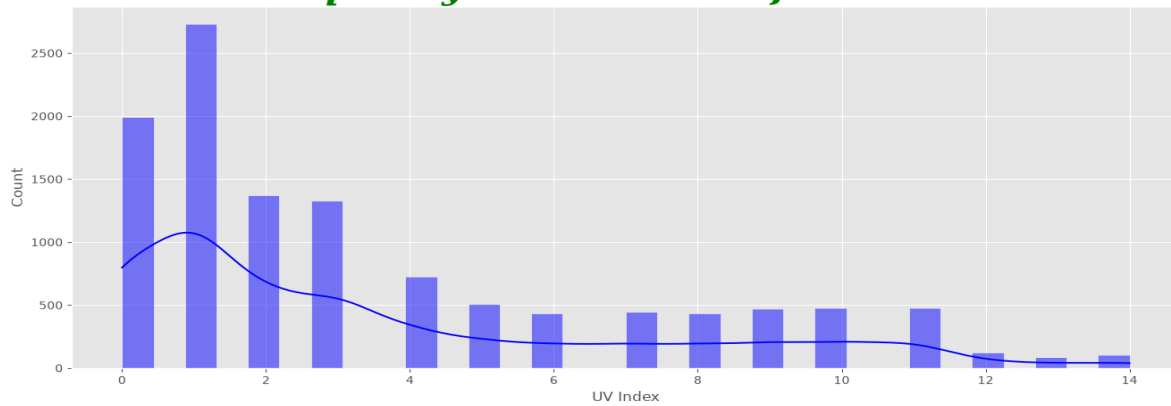
Frequency Distribution of Precipitation (%)



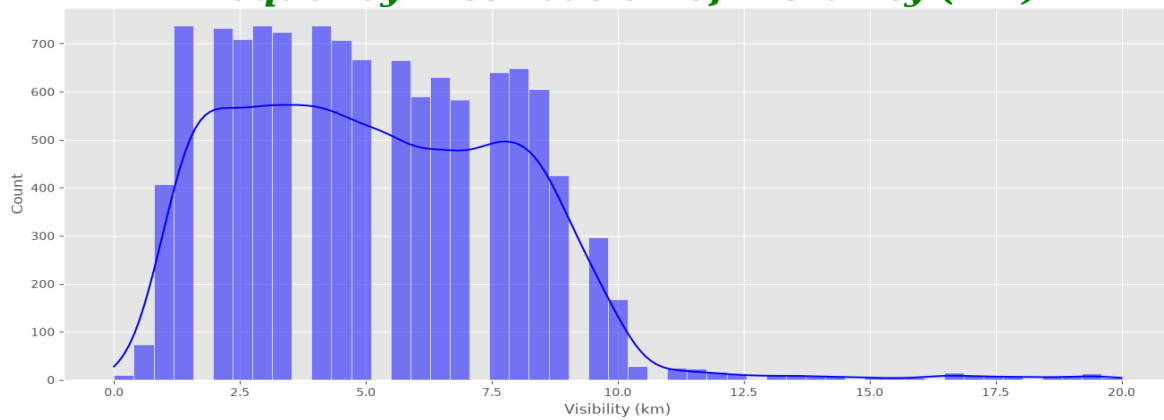
Frequency Distribution of Atmospheric Pressure (hpa)



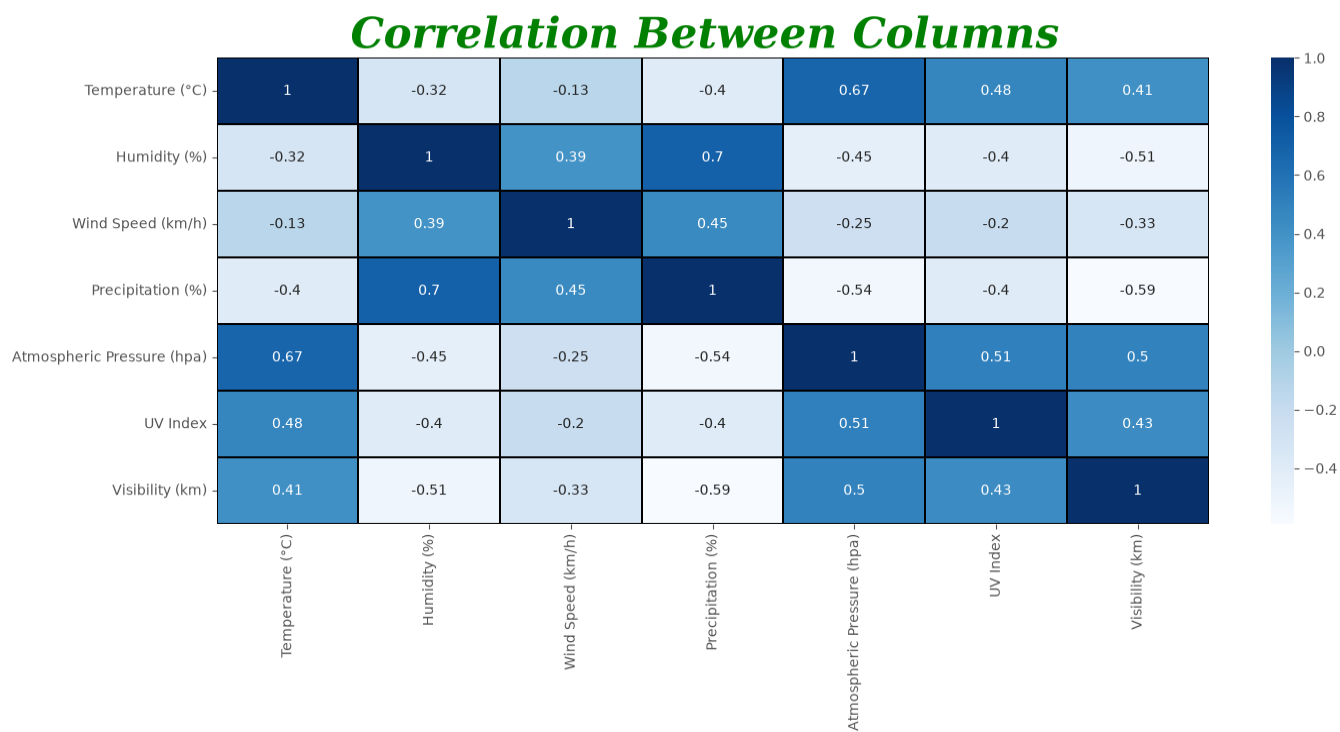
Frequency Distribution of UV Index



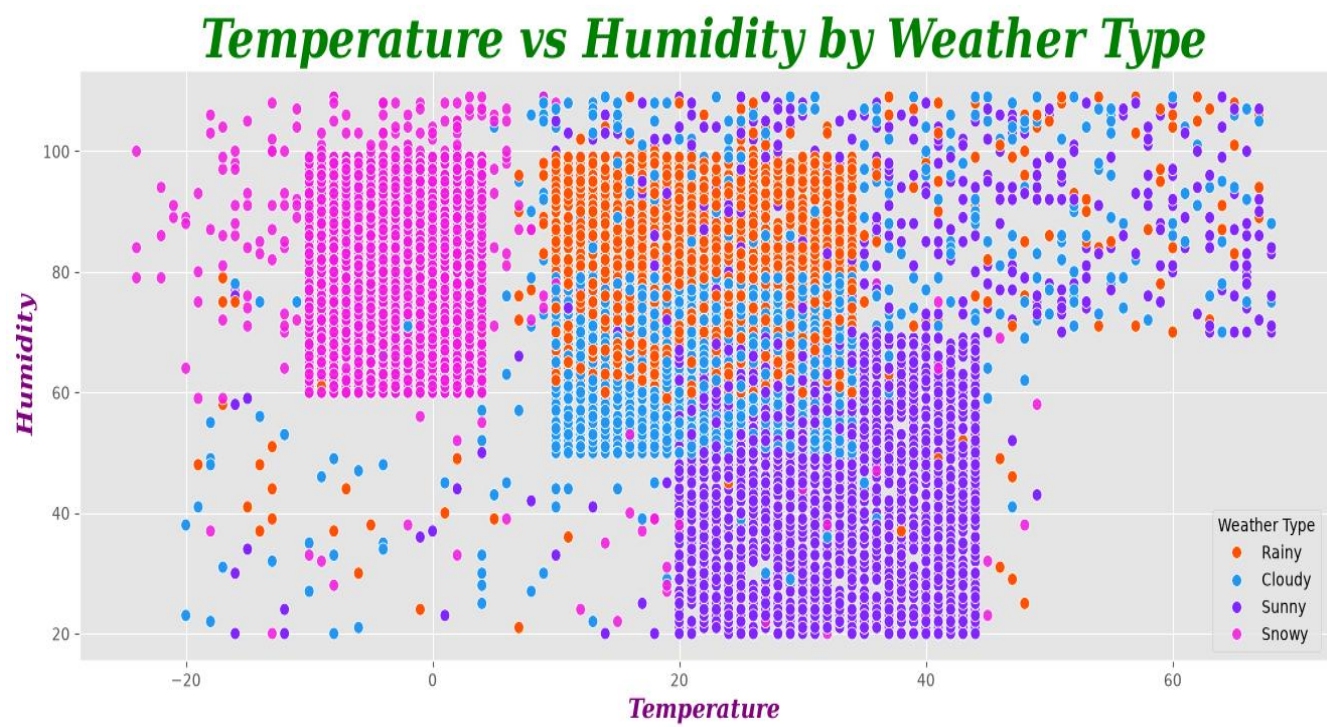
Frequency Distribution of Visibility (km)



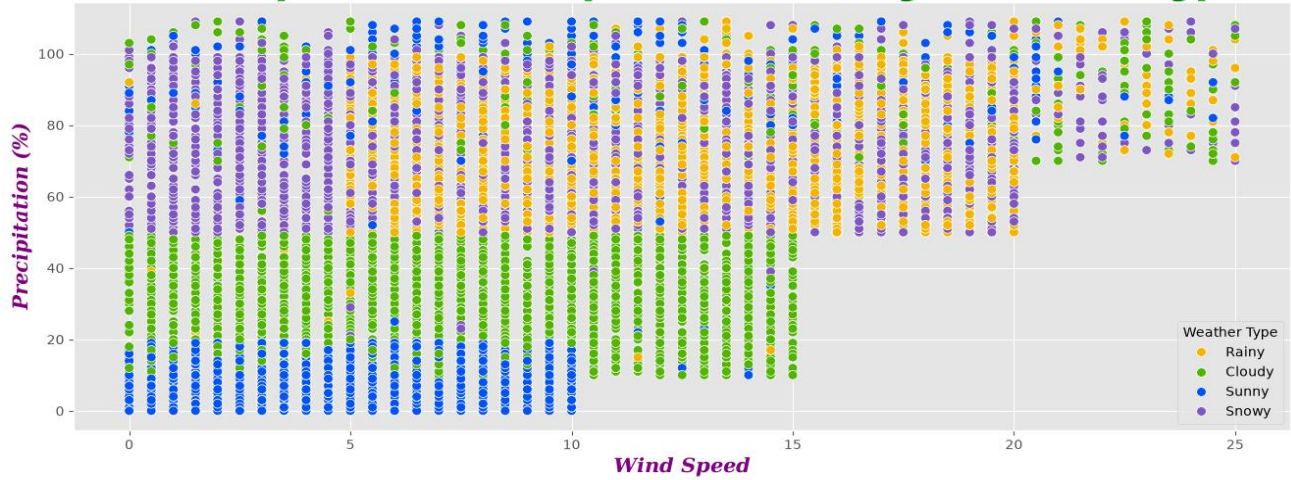
b. Correlation Between Numeric Features



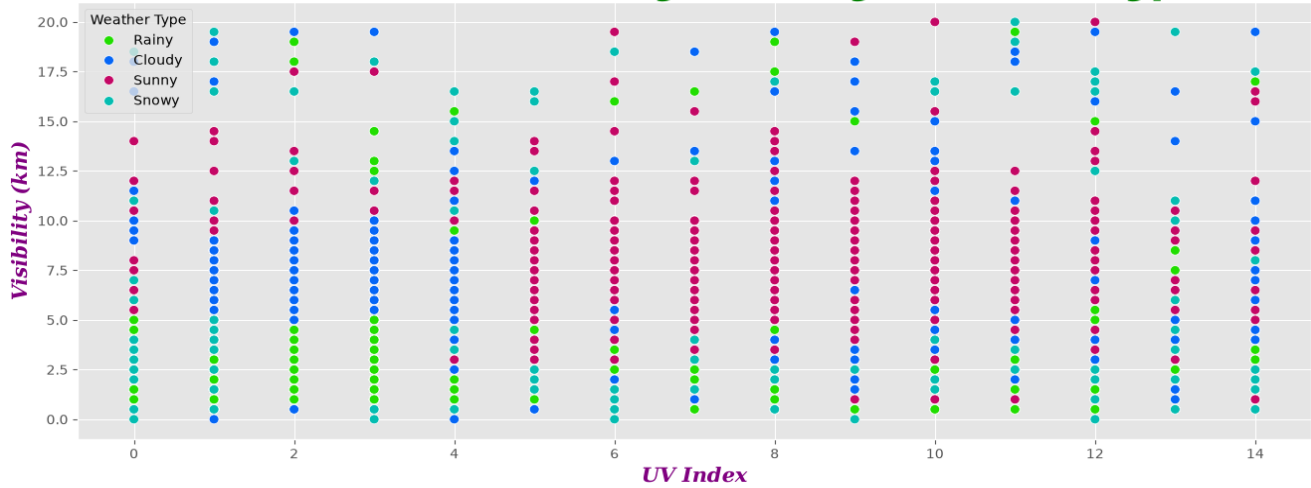
c. Relationship With Weather Type (Scatter Plots)



Wind Speed vs Precipitation (%) by Weather Type

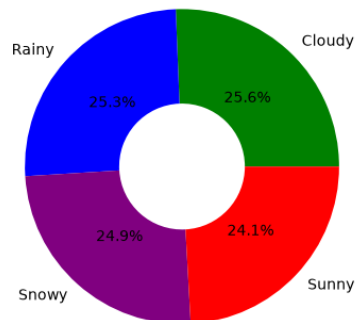


UV Index vs Visibility (km) by Weather Type



d. Overall Weather Type Distribution

Weather Type Distribution



The dataset is fairly balanced across the four Weather Type classes, which is confirmed later by very similar support counts in the classification report.

7. Classical Machine Learning — Model Building

Five different classical classification models are trained on the preprocessed (scaled + encoded) features to establish a baseline before moving to a deep learning approach :

- Decision Tree Classifier (max_depth=5)
- Logistic Regression()
- Random Forest Classifier (n_estimators=100, max_depth=10)
- Support Vector Machine — SVC (kernel='rbf')
- K-Nearest Neighbors (n_neighbors=5)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

dt_model = DecisionTreeClassifier(max_depth=5, random_state=98)
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale')
rf_model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=56)
knn_model = KNeighborsClassifier(n_neighbors=5, weights='uniform')
lr_model = LogisticRegression()

# each model is fit on x_train_scaled, y_train and evaluated on x_test_scaled
```

8. Artificial Neural Network (ANN) — Model Training

- The project builds a dedicated Artificial Neural Network model for non-linear feature interactions using a multi-layer architecture with dropout and L2 regularization, and this ANN is carried forward as the final model that is evaluated in depth and used for the final prediction .
- A feed-forward neural network is built with TensorFlow/Keras to classify the four weather types. The network uses progressively smaller Dense layers with ReLU activations, Dropout for regularization and L2 weight regularization to reduce overfitting, ending in a 4-unit Softmax output layer.

a. Network Architecture

Layer	Output Units	Activation	Regularization
Dense 1	64	ReLU	L2 (0.0001) + Dropout 0.1
Dense 2	32	ReLU	L2 (0.0001) + Dropout 0.1
Dense 3	16	ReLU	L2 (0.0001) + Dropout 0.1
Dense 4	8	ReLU	L2 (0.0001) + Dropout 0.1
Output	4	Softmax	L2 (0.0001)

Total trainable parameters: 3,548

b. Compilation & Training Configuration

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping

model = Sequential()
model.add(Input(shape=(x_train_scaled.shape[1],)))
model.add(Dense(64, activation='relu', kernel_regularizer=l2(0.0001)));
model.add(Dropout(0.1))
model.add(Dense(32, activation='relu', kernel_regularizer=l2(0.0001)));
model.add(Dropout(0.1))
model.add(Dense(16, activation='relu', kernel_regularizer=l2(0.0001)));
model.add(Dropout(0.1))
model.add(Dense(8, activation='relu', kernel_regularizer=l2(0.0001)));
model.add(Dropout(0.1))
model.add(Dense(4, activation='softmax', kernel_regularizer=l2(0.0001)))

optimizer = Adam(learning_rate=0.001)
model.compile(loss='sparse_categorical_crossentropy', optimizer=optimizer,
metrics=['accuracy'])

early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

history = model.fit(x_train_scaled, y_train, epochs=100, batch_size=32,
validation_data=(x_val_scaled, y_val), callbacks=[early_stopping])
```

- ✓ Early Stopping monitors validation loss and restores the best-performing weights, preventing the network from overfitting.
- ✓ Learning rate controls how quickly the network updates its weights during training, providing a balanced learning process and helping the model converge smoothly.

9. Model Evaluation

The trained ANN is evaluated in depth using train, validation and test scores, followed by a full classification report and confusion matrix.

a. Train / Validation / Test Scores

Dataset	Loss	Accuracy
Train	0.0815	97.37 %
Validation	0.1015	97.16 %
Test	0.1230	96.30 %

The train, validation and test accuracies stay very close to one another (97.4% / 97.2% / 96.3%), indicating the ANN generalizes well and is not overfitting.

b. Classification Report

Class	Precision	Recall	F1-score	Support
Cloudy (0)	0.96	0.92	0.94	451
Rainy (1)	0.95	0.96	0.95	424
Snowy (2)	0.98	0.99	0.99	438
Sunny (3)	0.96	0.99	0.97	446
Accuracy			0.96	1759
Macro Avg	0.96	0.96	0.96	1759
Weighted Avg	0.96	0.96	0.96	1759

c. Confusion Matrix

Actual \ Predicted	Cloudy	Rainy	Snowy	Sunny
Cloudy	416	23	3	9
Rainy	9	406	2	7
Snowy	3	0	432	3
Sunny	4	0	2	440

10. Model Accuracy Comparison

The test-set accuracy of every model — the five classical ML models plus the final ANN is summarized below:

Model	Type	Test Accuracy
Decision Tree	Classical ML	95.79 %
Logistic Regression	Classical ML	94.49 %
Random Forest	Classical ML	96.87 %
SVM (RBF Kernel)	Classical ML	96.25 %
K-Nearest Neighbors	Classical ML	95.51 %
Artificial Neural Network (ANN)	Deep Learning	96.30 %

Observation:

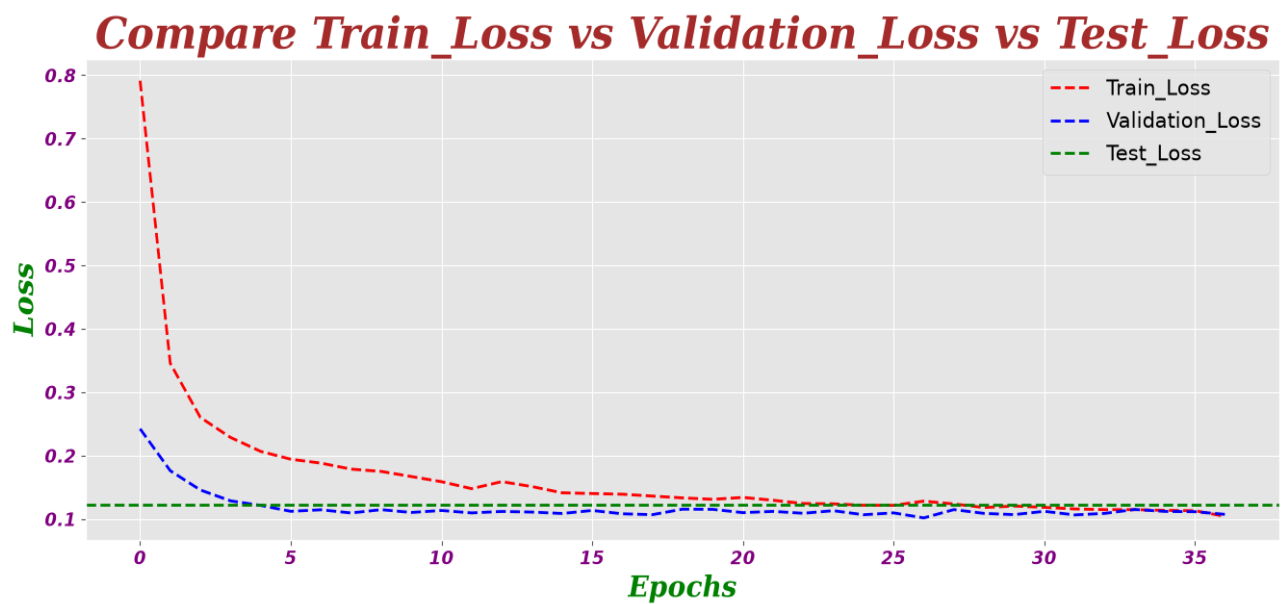
- ✓ All models score close to one another (94–97%), showing the features are strongly predictive of Weather Type.
- ✓ Final model Selected – Artificial Neural Network (ANN)

- ✓ The model comparison shows that all five classical Machine Learning models achieved strong test accuracy, The ANN achieved 96.30%, which is comparable to the classical models.
- ✓ ANN was selected as the final model because it can learn complex and non-linear interactions among multiple weather features through its multi-layer architecture. In addition, Dropout and L2 regularization were control overfitting, while Early Stopping was used to retain the best validation performance.

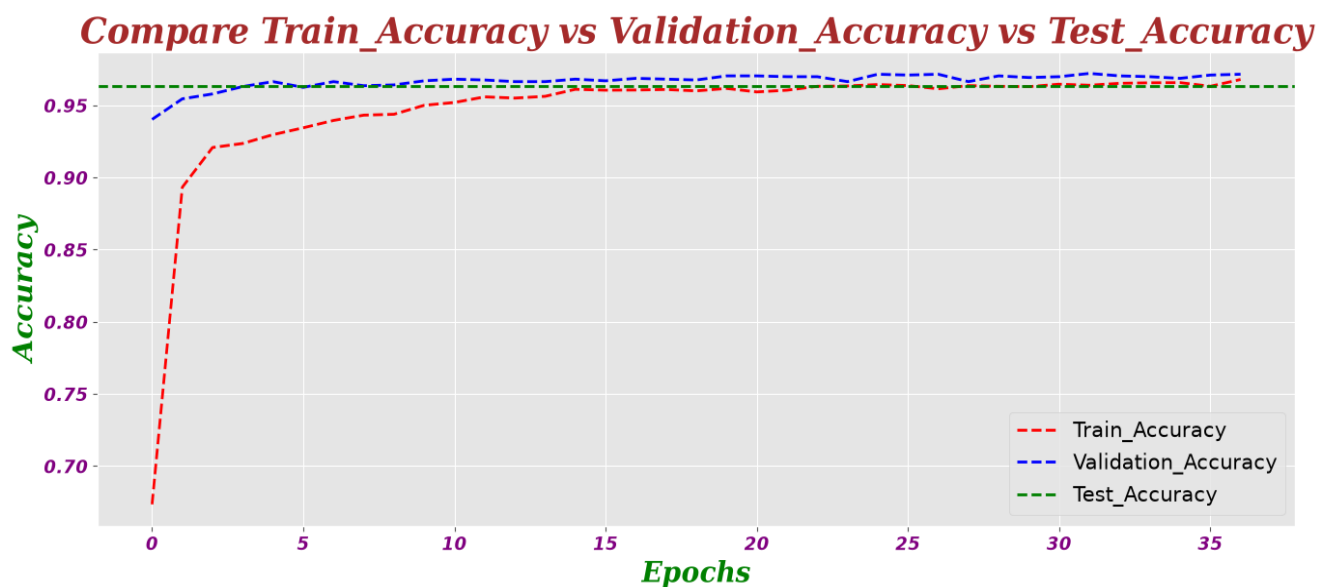
11. Final Model Performance Visualization

The final section visualizes the ANN training behaviour and compares actual vs predicted weather types on the test set.

a. Loss Curve

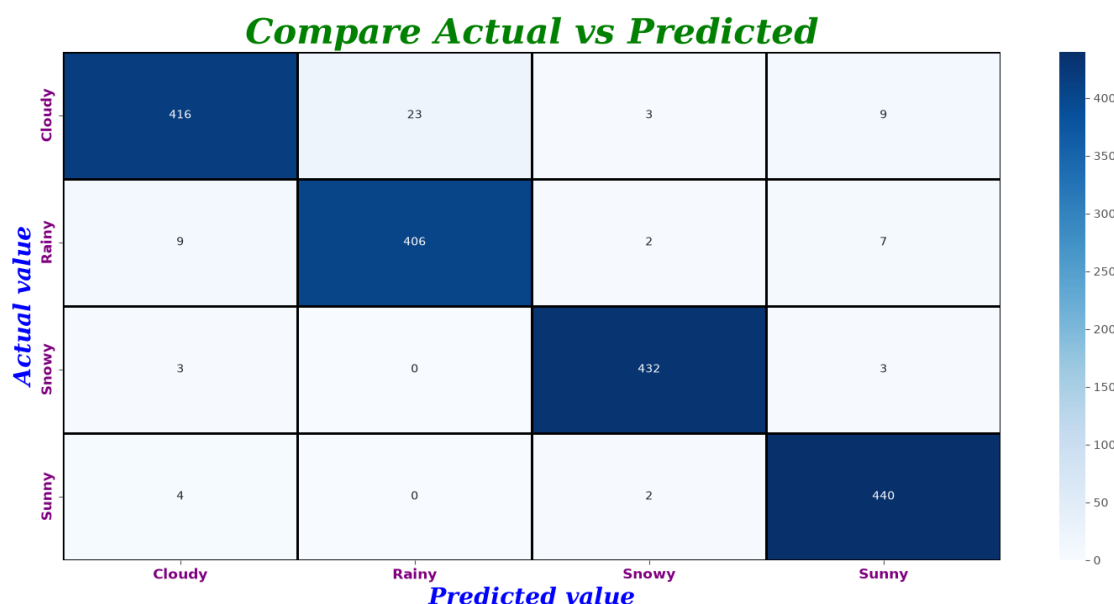


b. Accuracy Curve



Observation : The loss curves decrease and stabilize, while the accuracy curves increase and converge at a consistently high level. The close alignment between training ,validation and testing curves indicates stable learning with no significant overfitting. The final train, validation and test accuracies of 97.37%, 97.16% and 96.30% further confirm the model's consistent generalization performance.

c. Actual vs Predicted (Confusion Matrix Heatmap)



Observation : The diagonal values (416, 406, 432, 440) represent correctly classified samples out of a 1,759-sample test set, confirming strong predictive performance with only minor confusion mainly between Cloudy and Rainy classes.

12. Conclusion

- This project demonstrates an end-to-end workflow for predicting weather type — from raw data to a deployable deep learning model.
- IQR-based outlier removal and a structured Scikit-Learn Pipeline with a ColumnTransformer were used to clean, scale and encode features consistently.
- A custom Artificial Neural Network with Dropout and L2 regularization was trained and selected as the final model, achieving ~96.3% test accuracy with train/validation/test scores staying close together, confirming no significant overfitting.
- The project highlights the value of comparing classical ML baselines against a tuned deep learning model before finalizing a deployment-ready classifier.

Thank You